

NOTES-RobustArithmetic

QuantDialectics — librería del Protocolo Empírico Trinitario

José Mauricio Gómez Julián

2026-08-20

Índice

RobustArithmetic — bitácora de construcción	1
§1. Sesión 79 — fases 0 y 1, con sus dos portones cruzados	2
§1.1. Fase 0 — esqueleto, tabla cerrada, sondas portadas	2
§1.2. La lectura de la fase 1, y lo que cambió por leerla	3
§1.3. El texto del estándar CORRIGE un veredicto del Bloque V	4
§1.4. Fase 1 — el núcleo inf-sup	4
§1.5. Portón 1 — el núcleo contra MPFR a 300 bits	5
§1.6. Estado al cerrar la 79	6
§2. Sesión 80 — fases 2 y 3, con sus dos portones, y el ancla del criterio C3 corregida . .	6
§2.1. El paso 0, y la re-auditoría que sí encontró algo	6
§2.2. Fase 2 — las elementales de dos niveles	6
§2.3. El hallazgo de fondo: el criterio C3 estaba anclado a una rutina que R no llama	7
§2.4. La enmienda §4-bis: la frontera periódica la fija el formato, no la reducción .	8
§2.5. La escalada, con su abstención	8
§2.6. Fase 3 — expresiones, y la dependencia que no se tapa	8
§2.7. Portón 2 — las elementales contra MPFR a 300 bits, 10^6 puntos por función	9
§2.8. Portón 3 — las expresiones contra su propio rango, muestreado denso	9
§2.9. Estado al cerrar la 80	10

RobustArithmetic — bitácora de construcción

Paquete separado nacido de la pausa comisionada de QuantDialectics (sesión 78 diseña, 79 ejecuta). Gobiernan `~/.qd_s78_work/BLOQUE_V_ARITMETICA_INTERVALAR.md`, `BLOQUE_VI_LECTURA_DE_MAPA.md` y `PAUTAS_EJECUCION_OPUS.md`. Cero commits hasta orden del autor.

§1. Sesión 79 — fases 0 y 1, con sus dos portones cruzados

§1.1. Fase 0 — esqueleto, tabla cerrada, sondas portadas

Esqueleto: ~/RobustArithmetic (ext4), License: GPL (≥ 3) con LICENSE.md en .Rbuildignore, Roxygen2 8.0.0 con el set completo de tags en cada exportada, testthat edición 3, Imports: stats, Suggests: Rmpfr, testthat, knitr, rmarkdown.

La tabla de operadores, cerrada por CUATRO criterios y no por uno. Las pautas pedían enumerar contra el `stats::D` instalado; eso es condición necesaria y no suficiente. Los cuatro, todos medidos contra el software instalado (`dev/measure_statsD.R` y `dev/close_operator_table.R`, con sus salidas al lado):

- **C1** — el `stats::D` instalado lo deriva.
- **C2** — la tabla es **CERRADA bajo diferenciación**: la derivada de un símbolo admitido, y la de ésa, hasta punto fijo, se escribe con símbolos admitidos. Es el criterio que una tabla armada por gusto falla, y fallarlo no es una molestia sino un agujero: la forma del valor medio de la fase 3 se sale de la tabla en el primer paso y no hay qué evaluar.
- **C3** — existe error máximo conocido en ulps para la libm de esta máquina (glibc 2.43) en el reporte de Gladman, Innocente, Mather, Ozaki y Zimmermann, edición de febrero de 2026, Tabla 3. Sin ese número no hay holgura declarable.
- **C4** — MPFR la provee correctamente redondeada, para que el nivel riguroso y la escalada tengan a dónde ir.

La tabla v1, 16 funciones: `exp log sin cos tan sinh cosh tanh sqrt expm1 log1p log2 log10 asin acos atan`. Más los operadores `+` `-` `*` `/` `^` `(` y el menos unario. **La clausura del conjunto es el propio conjunto** — verificado, y re-verificado en cada corrida por `ra_verify_operator_table()` y por un caminante independiente escrito en el test.

Las exclusiones, cada una con el criterio que falla y su medición:

- **La familia gamma** (`gamma`, `lgamma`, `digamma`, `trigamma`, `psigamma`) sale por **C4 MEDIDO**: el Rmpfr instalado contesta «Math op. 43 not yet implemented» a `trigamma`, y la cadena `gamma` $\rightarrow$ `digamma` $\rightarrow$ `trigamma` la alcanza en la tercera derivada. Nótese que C2 pasa formalmente (la derivada de `psigamma` es `psigamma` de orden mayor: el SÍMBOLO cierra), pero el orden crece sin techo y ningún conjunto finito de funciones cierra la familia.
- **pnorm/dnorm** salen por **C3**: R las computa en su propia biblioteca numérica y no en la libm, así que la Tabla 3 no dice nada sobre ellas y no hay holgura declarable sobre un error que nunca se midió. Puerta nombrada: si alguna vez se mide contra `Rmpfr::pnorm`, entran.
- **abs sign asinh acosh atanh floor ceiling round trunc max min** salen por **C1**.

Un hallazgo que corrige una sospecha propia: creí que `stats::D` no traía la regla general de la potencia (que `D(x^y, "x")` daba sólo `x^(y-1)*y`). **Es falso:** con exponente dependiente de `x` da los dos términos y es correcta ($D(x^x) = x^{(x-1)}x + x^x \log(x)$), verificado contra la derivada verdadera y contra la diferencia central). La medición cerró la duda; no hay guarda que escribir.

Las holguras `slack(f) = ceiling(2*e(f) + 1)`, con las cinco cifras del Bloque V confirmadas contra el reporte y once más añadidas: `exp/log/sin/cos/tan/expm1/log1p/log2/asin/ acos/atan` **3 ulps**; `sinh/cosh/log10` **5**; `tanh` **6**; `sqrt` **2**. La entrada de `sqrt` es deliberadamente conservadora: su 0,500 es cota SUPERIOR por teorema (IEEE 754-2019 §5.4.1 exige redondeo correcto de la raíz), no inferior como el resto; se le aplica igual la convención uniforme para que ninguna función lleve regla propia.

Las dos sondas portables, hechas tests permanentes con sus controles positivos: `test-rounding.R` (sonda 2) y `test-mpfr-bridge.R` (sonda 1b). Los CP disparan: la fórmula ingenua $\phi = 2^{-53}$ viola validez en $x = 1$; quitar el término ϵ rompe en 0; `as.numeric` subdesborda a 0 donde `toNum` da 2^{-1074} .

Portón 0 cruzado: R CMD check natural **Status: OK.**

§1.2. La lectura de la fase 1, y lo que cambió por leerla

RZBM 2009, entero. El Teorema 2.1 da la validez ($\text{cinf} \leq \text{pred}(c)$, $\text{succ}(c) \leq \text{csup}$) para $c \in \mathbb{F}$ finito. El Teorema 2.2 da algo que la sonda 78 había medido y no sabía que era un teorema: la igualdad vale para todo c finito fuera de la franja $|c| \in [u^{-1}\eta/2, 2u^{-1}\eta]$, que en `binary64` son las dos binadas $[2^{-1022}, 2^{-1020}]$, y dentro de la franja el resultado está a un bit. Ambos teoremas están verificados en Coq.

Lo verifiqué en esta máquina y da exactamente eso: de 180 092 puntos, **8 no exactos, y los ocho en 2^{-1022} y 2^{-1021} y sus negativos** — todos dentro de la franja, ninguno fuera. La aserción del test pasó de estadística («99,99% exactos») a **estructural** («todo punto no exacto cae en la franja que el teorema excluye»), que es la que puede fallar informativamente: una proporción puede seguir alta mientras las excepciones se mudan a donde el teorema las prohíbe.

Otro punto del papel que licencia el diseño: la nota a (1.5) dice que el encierro vale «si $a \circ b$ se reemplaza por cualquier real y , por ejemplo $y = \sin x$, **mientras c sea el redondeo correcto de y** ». Ésa es exactamente la condición que la libm NO cumple y la razón por la que existe la holgura del nivel rápido.

Neumaier §5.6 (y 5.1.8–5.1.10). Tres cosas:

1. **El paso 3 del algoritmo de bisección de Neumaier ES la verificación de candidato por inflado:** «tratar de verificar la existencia de una solución en una caja angosta $x' \supseteq x$ por los métodos de §5.5». La sonda 4 de la 78 lo descubrió por su cuenta al fallar 12 de 27; está en el algoritmo de referencia desde 1990. La reparación no era una ocurrencia: era la canónica, y ahora tiene cita.
2. **El Teorema 5.1.8(iii) NO pide $0 \notin f'(x)$ como hipótesis:** pide $\check{x} \in \text{int}(x)$ y $0 \neq H(\check{x}, x) \subseteq \text{int}(x)$, y de ahí DERIVA que A es fuertemente regular. La formulación del Bloque V (que añade $0 \notin f'(x)$) es suficiente y no mínima; en una dimensión son operacionalmente equivalentes. Anotado para la fase 4: la hipótesis se implica, no se asume.
3. **El paso 5 parte la caja de modo que el minimizador aproximado quede en la parte MAYOR,** y la menor va a la pila. El V.5 dice «bisección al punto medio», que es válido; el algoritmo de referencia es más fino. Anotado para la fase 4.

Hansen–Walster §9.2. Las ecuaciones (9.2.3) y (9.2.4) son la tabla de casos exacta de la división extendida del paso de Newton, con sus tres ramas ($c = 0$, $d = 0$, $c < 0 < d$). Implementada tal cual en `.ra_div_pieces()`.

Rump *Acta Numerica* §§1–4. El §4 trae **una trampa que toca a la semilla de QD:** el Teorema 4.2 «no es válido si se reemplaza $R^*A - \text{eye}(n)$ por $\text{eye}(n) - R^*A$; en ese caso la multiplicación y la resta deben computarse en modos de redondeo OPUESTOS». La `ivl_verified_inverse` de QD usa justamente la forma $I - RA$ (`id - ax$hi`, `id - ax$lo`) y la resuelve intercambiando extremos, que es la forma intervalar de la prescripción — pero cuando esa capa reciba redondeo hacia afuera, ahí está la trampa esperando. Anotado.

§1.3. El texto del estándar CORRIGE un veredicto del Bloque V

Portón de lectura de V.6 cumplido sobre IEEE Std 1788.1-2017 (cláusulas 4.2, 4.5.1–4.5.4, 5.2–5.7).
Manda la letra, y la letra dice otra cosa que el Bloque V en un punto:

V.6 decía: «los errores de dominio (log de intervalo que toca ≤ 0 , etc.) con `ill` y su condición tipada».

La cláusula 5.3 dice: `ill` surge **si y sólo si** (a) un constructor no puede construir un intervalo decorado válido, o (b) una operación aritmética de la biblioteca recibe una entrada mal formada. **Un error de dominio NO es `ill`:** es que `def` falla, y la decoración más fuerte que queda es `trv`.

Aplicado: `log([-1,2])`, `1/[-1,1]`, `x^(-1)` sobre intervalo que contiene 0 — todos `trv`, ninguno `ill`. `ill` queda para `NaN`: construcción inválida y su propagación.

Otras tres cosas de la letra que el Bloque V no decía y que ahora están:

- **newDec (5.5.1):** `com` si es no vacío y **acotado**; **dac si es no acotado**; `trv` si es vacío. Un intervalo no acotado NO puede llevar `com`.
- **Combinaciones prohibidas (5.4):** $\emptyset$ con `def/dac/com`, y `com` con no acotado. `setDec` las **repara** en lugar de rechazarlas o producirlas.
- **setDec (5.5.2) es obligatoria** y no estaba en el diseño. Escrita: `ra_set_dec()`. La destapó un test que quería construir una combinación prohibida y no tenía por dónde.
- **Regla del mínimo (5.6):** `dw = min{dv0, dv1, ..., dvk}` en el orden `com > dac > def > trv > ill`, con `dv0` la decoración local de la operación.
- **Operaciones no aritméticas (5.7.1):** `intersection` y `convexHull` se decoran `trv`.
- **Cláusula 4.2:** los infinitos son COTAS y nunca miembros, así que `[+∞, +∞]` y `[-∞, -∞]` no son intervalos no vacíos. El constructor los rechaza.

Una divergencia declarada (el paquete reclama «conformidad parcial, subconjunto nombrado»): el constructor **levanta** condición tipada donde el estándar dice que el constructor decorado devuelve `NaN`. En R, un contrato violado en construcción debe frenar al llamador y no viajar como valor; la forma valor queda disponible en `ra_nai()`.

Y una conciliación, no una cesión: las pautas piden que «la división por intervalo que contiene 0 por la vía NO extendida DEBE rechazar», y el estándar pide que `div` sea total y devuelva el casco con `trv`. Se cumplen las dos: `/` y `ra_div()` siguen al estándar (casco + `trv`, que es la respuesta más ancha con la decoración más débil — grita, no susurra), y `ra_div(x, y, zero = "error")` levanta `ra_division_straddles_zero`. Ninguna de las dos miente; el control positivo prueba las dos.

§1.4. Fase 1 — el núcleo inf-sup

Clase `ra_ivl`: tres vectores paralelos (`lo`, `hi`, `dec`), vectorizada. **Vacío** = par invertido (`+Inf`, `-Inf`), de modo que la vacuidad es `lo > hi` y la intersección es `(max lo, min hi)` sin análisis de casos. **NaN** = `(NaN, NaN)` con `ill`.

Escrito: constructor con su invariante verificado UNA vez; `ra_empty/ra_entire/ra_nai`; accesores; `ra_wid/mid/rad/mag/mig`; `Ops.ra_ivl` (+ - * / y menos unario, con comparaciones **rechazadas** porque el orden que sugerirían no es el de los conjuntos); `ra_neg/add/sub/mul/div`; `ra_div_extended`; `ra_sqr/ra_abs/ra_pown`; `ra_intersect/ra_hull`; `ra_set_dec`. Y todo lo que arrastra la clase,

JUNTO (§203.7 de QD): `format`, `print`, `as.data.frame`, `summary` con su clase y su impresión, `length`, `c`, `[]`.

Dos defectos propios encontrados y reparados en el camino:

1. **ra_pown heredaba el trv de la intersección.** El paso de estrechamiento interno usa `ra_intersect`, que el estándar decora `trv` por ser operación no aritmética — correcto para la operación y equivocado para este resultado, que sí es extensión intervalar de una función puntual. Se le calcula la decoración propia.
2. **.ra_pown_tight aplicaba UN paso de redondeo a $x \cdot 10^q$.** El $\wedge$ de R con exponente entero multiplica repetidamente (`R_pow_di`), así que su error crece con el exponente y un paso deja de cubrirlo arriba del cubo. **El portón 1 lo cazó:** `pow3` violó contención contra MPFR. Reparado por construcción: exponenciación binaria sobre INTERVALOS, donde cada intermedio lleva su propio paso y el resultado es encierro sin suposición de exactitud ninguna.

Un hallazgo cuantitativo de esa reparación: el ensanchamiento de `pown` crece lineal en `p` aunque las operaciones crezcan como $\log_2(p)$. Medido: 1 paso en `p=2`, 3 en `p=3`, 3 en `p=4`, 5 en `p=5`, 9 en `p=8`, 15 en `p=16`. La razón es que el error relativo de cada operación se lleva a la siguiente y se multiplica por ella, de modo que en ulps **del resultado** la cuenta es $\approx p \cdot u$ y no $\approx \log_2(p) \cdot u$. Documentado en el operador, y el test asiente `|p| + 1` en vez de la cota logarítmica que yo había escrito primero y que era falsa.

§1.5. Portón 1 — el núcleo contra MPFR a 300 bits

`tests/testthat/test-gate-kernel-vs-mpfr.R`. Referente externo: MPFR a 300 bits, que no comparte con el núcleo ni código, ni convención de redondeo, ni representación.

- **Contención, aserción absoluta:** 10^5 pares aleatorios por operador (magnitudes de 2^{-40} a 2^{40} , ambos signos, con casos incómodos inyectados: extremos en cero, intervalos que cruzan el cero, intervalos punto). El resultado puntual de alta precisión SIEMPRE adentro. Idem para `sqr`, `abs`, `neg`, `pown` con `p` $\in \{2, 3, 5\}$ sobre $3 \cdot 10^4$.
- **Estrechez, reportada y no enmascarada** — exceso en pasos (inferior + superior) contra el óptimo, que es el par de dobles más ajustado que contiene el rango exacto:

operación	n	media	q99	máx
add	100 000	1,0527	2	2
sub	100 000	1,0646	2	2
mul	100 000	1,0497	2	2
div	46 098	0,9992	2	2

El portón pedía « ≤ 2 ulps del óptimo en el 99 %»: **el máximo es 2, no el percentil 99.**

- **El óptimo mismo queda encerrado**, aserción absoluta aparte: el núcleo nunca es más estrecho que el par de dobles más ajustado válido.
- **Control positivo:** reemplazar el paso hacia afuera por uno hacia adentro DEBE romper la contención. Dispara (5 000 puntos, violaciones > 0), y el núcleo verdadero no viola sobre los mismos datos.

Portón 1 cruzado. Suite: **PASS 330 · FAIL 0 · WARN 0 · SKIP 2 · ERROR 0** (los dos SKIP son ramas inalcanzables con `Rmpfr` instalado: lo que el paquete hace SIN el backend).
R CMD check natural: **Status: OK.**

§1.6. Estado al cerrar la 79

- **Fases 0 y 1 TERMINADAS, con sus dos portones cruzados.** Pendiente: fases 2, 3, 4 y 5.
 - **Cero commits**, en RobustArithmetic y en QD. QD sin tocar: ni R/, ni la suite.
 - Espejo a Kingston: **NO movido todavía** — se mueve terminado, y no lo está.
 - Lo que la fase 2 hereda escrito y medido: la tabla con sus 16 funciones y sus holguras, la escalera {106, 212, 424, 848}, el puente `ra_to_double/ra_enclose_mpfr` medido, y el núcleo entero sobre el que apoyar las elementales.
-

§2. Sesión 80 — fases 2 y 3, con sus dos portones, y el ancla del criterio C3 corregida

Pre-registro de la fase 2 en `dev/PREREGISTRO_FASE_2.md`, escrito antes de la primera línea de `R/elementary.R`, con **dos enmiendas fechadas** que se explican solas más abajo. Cero commits.

§2.1. El paso 0, y la re-auditoría que sí encontró algo

Versiones confirmadas (R 4.6.1, Rmpfr 1.1.2, testthat 3.3.2, roxygen2 8.0.0, ED 0.1.10 sin Repository, gdpar 0.0.0.9001, topologyR 0.3.0); R/ ASCII puro en QD y acá; suite de arranque **PASS 330 · FAIL 0 · WARN 0 · SKIP 2 · ERROR 0**, la cifra exacta de la 79. `ra_verify_operator_table()` da `closed = TRUE` con los cuatro criterios vacíos, y su control positivo dispara (admitir γ a la fuerza hace escapar digamma , psigamma , trigamma).

Y acá la re-auditoría dejó de ser un trámite. La regla dice re-auditar las **premisas**, no sólo las cifras. La premisa de la 79 era «las cinco holguras del Bloque V confirmadas contra el reporte y once más añadidas». El reporte **no estaba en disco**: no estaba en Robust Arithmetic/Literatura/, que tenía 17 documentos y ninguno era éste. Bajado y depositado ahí (ahora son 18). Verificadas **las 16 cifras contra la Tabla 3, columna GNU libc 2.43, edición de febrero de 2026**: coinciden dígito a dígito, las 16. Los cinco autores son los citados, Ozaki incluido.

Y la lectura trajo un dato que la bitácora no tenía y que la fase 2 necesitaba: **el §3.1 recorre el espacio ENTERO de binary64 mapeado a uint64_t**, y el texto nombra la reducción de argumento cerca de 2^{1024} para seno y coseno como algo que el buscador debe detectar solo. Sin eso, la holgura de $\sin/\cos$ tendría un agujero justo donde la fase 2 la apoya.

§2.2. Fase 2 — las elementales de dos niveles

`R/elementary.R`. Cinco clases de forma, declaradas y **verificadas contra las propias funciones** por `ra_verify_monotonicity()`, que lee los signos de las diferencias sucesivas sobre una malla del dominio y distingue polo de período por la longitud de la corrida más corta: un polo invierte el signo durante un paso, un semiperíodo durante cientos. Su control positivo es declarar `cosh` creciente a propósito, y aparece en `disagreeing`.

La búsqueda de extremos interiores es lo que la fase pedía de verdad. Los puntos críticos de $\sin/\cos$ y los polos de $\tan$ se escriben todos como $q\pi + k(m\pi)$, y la pregunta es si el intervalo de índices $K = (x - q\pi)/(m\pi)$ contiene un entero. K se computa con π **encerrado**, de modo que es SUPERCONJUNTO del verdadero: **puede ver un extremo que no está — ensancha— y no puede perder uno que sí está**, que es el único modo de falla que rompería

la contención. Un intervalo degenerado se salta la búsqueda por teorema: **ningún doble es un polo, porque todo polo es irracional.**

Tres defectos propios, encontrados y reparados:

1. **La holgura empujaba el extremo fuera del RANGO de la función.** `sqrt([0,4])` daba `[-2^-1074, 2]`: cota inferior negativa para una función que nunca es negativa. Válido y engañoso. Reparado interseccionando con el rango, con los límites irracionales guardados ya redondeados hacia afuera — **el doble más cercano a $\pi/2$ está por DEBAJO de $\pi/2$, así que recortar `asin(1)` contra él cortaría el valor mismo.**
2. **La regla local de decoración de la aritmética no sirve para las elementales.** `.ra_local_total()` mira sólo la salida, y `atan([-Inf, Inf])` sale acotado desde una entrada que no lo es; la cláusula 5.5.1 pide que com afirme que la caja de ENTRADA es acotada. Regla propia escrita.
3. **elementary.R se carga antes que rounding.R** (orden alfabético), así que una tabla de rangos guardada como vector de nivel superior llamaba a `ra_succ()` inexistente y el paquete no cargaba. Es la tercera trampa silenciosa de R después de las dos ya anotadas: **el código de nivel superior no puede suponer el orden de lectura de los fuentes.**

§2.3. El hallazgo de fondo: el criterio C3 estaba anclado a una rutina que R no llama

Lo destapó el **margen** del portón 2, que existe justamente porque un portón que pasa sin decir por cuánto no distingue una holgura ajustada de una absurda.

Primero hubo que reparar el instrumento del margen. La primera versión medía el error en binary64 y salía cuantizado en enteros — 1, 2 —, o sea **más grueso que todas las cifras que tenía que resolver**, que van de 0,500 a 2,21. Es la regla del censo medido con instrumento más angosto que su objeto, en su forma opuesta. Se mide en MPFR y ahí resolvió fracciones.

Con el instrumento reparado, **seis funciones dieron error observado por encima del publicado** (exp 1,279 contra 0,511; cos 1,184 contra 0,516; sin 0,857 contra 0,516; expm1, tan, atan). Que 20 000 puntos al azar le ganen a una búsqueda dirigida sobre 2^{64} entradas es **implausible**, y esa implausibilidad obligó a investigar en vez de anotar.

Aislado fuera de R. Un programa en C con gcc -02 que lee el argumento de argv — para que no haya plegado de constantes — llama a `sin(0x1.b981319337b63p-26)` y obtiene `0x1.b981319337b62p-26`, el correctamente redondeado (verificado en MPFR y por serie de Taylor a 500 bits, que coinciden a 4e-34 ulp). **R devuelve ...b63**, un ulp arriba. Es decir: **R y la libm escalar de glibc 2.43 no dan lo mismo, y el que se aparta del correctamente redondeado es R.**

Mi primer diagnóstico fue FALSO y queda escrito. Dije «vector de largo ≥ 2 va a libmvec». Con `R_ENABLE_JIT=0` todos los caminos dan ...b63, incluido el de largo 1; con el JIT en su valor por defecto, el único camino que da ...b62 es una llamada a `sin` sobre un vector de largo 1 **dentro de código compilado**, que es un atajo escalar del compilador de byte-code. No es cuestión de largo de vector: es **ruta de evaluación**, y depende de si el código está compilado. Y el atajo **no se alcanza** ni desde `vapply` ni desde `do.call` — medido, después de haber escrito una «reparación» con `vapply` que no reparaba nada y que se revirtió.

Qué se hizo y qué NO se hizo:

- **No** se agrandó ninguna holgura. Las 16 quedan como las declaró la 79.
- **No** se escribió código con la forma que dispara el atajo del compilador: sería depender de un interno no documentado, la clase de dependencia que deja de valer sin avisar.

- **Sí** se corrigió el ANCLA declarada. C3 decía «existe error máximo conocido en ulps para la libm de esta máquina»; ahora dice que esa cifra es ancla para una **rutina** y no certificado para la **ruta** que R usa, y que la ruta se **mide**. `ra_measure_library_error()` es esa medición, exportada, y el portón asiente que la holgura declarada la cubre.
- **Sí** quedó test permanente de que la ruta en uso NO es la correctamente redondeada, escrito contra el valor de MPFR y no contra ningún interno de R.

Es la regla de apuntar el instrumento a su propio producto: el paquete venía comparándose contra el reporte y **nunca contra su propia ruta de evaluación**.

§2.4. La enmienda §4-bis: la frontera periódica la fija el formato, no la reducción

Escrita **antes** de correr nada, al derivar la aritmética para escribir la pieza. El pre-registro decía que la frontera de resolución de `sin` sobre cajas grandes la pone la precisión de la reducción de argumento. **Es falso**. $K = X/(2\pi) - q/m$ tiene dos fuentes de ancho: el ancho de X dividido por 2π , y el redondeo de la división, del orden de $(a \cdot 2^{-52})/(2\pi)$. Pero $a \cdot 2^{-52}$ es exactamente un ulp de a , y **ningún intervalo no degenerado es más angosto que un ulp**. La segunda fuente nunca supera a la primera por más de un factor chico: **manda la granularidad de binary64**. Por encima de $2^{52} \cdot 2\pi \sim 2^{54,65}$, dos dobles vecinos están a más de un período uno de otro y $[-1,1]$ no es una pérdida: es la respuesta exacta y estrecha.

Predicción falsable pre-registrada y **cumplida**: la frontera medida para ancho relativo r cae en $\text{ceiling}(\log_2(2\pi/r))$ con tolerancia de un exponente, y **el nivel riguroso no la corre** (diferencia en $\{0,1\}$). Medido con $r = 2^{-20}$: predicho 23, rápido 23, riguroso 23. Si el riguroso la hubiera corrido mucho, la derivación sería falsa; si el rápido cayera muy por debajo, la reducción rápida estaría peor de lo que la derivación admite. **El criterio puede fallar por los dos lados**.

§2.5. La escalada, con su abstención

`ra_elem_escalate(fun, x, decide, ladder)` sube $\{106, 212, 424, 848\}$ hasta que el criterio del llamador se cumple, y en el tope devuelve la palabra «**no verdict at this budget (848 bits)**» con la cifra adentro. El criterio es del llamador y **nunca se afloja acá**. Objeto `ra_escalation` con todo lo que arrastra una clase, JUNTO: `format, print, as.data.frame, summary` con su clase y su impresión.

§2.6. Fase 3 — expresiones, y la dependencia que no se tapa

`R/expression.R`. Extensión natural sobre el árbol; símbolo fuera de la tabla cerrada $\rightarrow$ `ra_symbol_not_in_table` **con la razón por la que está excluido**; variable sin valor $\rightarrow$ `ra_unbound_variable`, nunca un NA silencioso. Exponente entero constante por `ra_pown` (estrecho, y aguanta base que cruza el cero); cualquier otro por `exp(y*log(x))`, que necesita base positiva y lo dice **por decoración**, no por error.

Tres rutas, y se INTERSECAN en vez de elegir. Natural; forma del valor medio $f(c) + f'(X)(X-c)$ con f' de `stats::D`; y monotonía cuando θ no está en $f'(X)$. Las tres son teoremas, la intersección de encierros válidos es válida, y **no hay ninguna decisión que auditar**: una regla que eligiera «la más angosta» estaría decidiendo por el llamador. Cuáles contribuyeron viaja en el atributo `ra_routes` y no se infiere del ancho.

Control positivo del diseño, cumplido: **$x - x$ sobre $[-1,1]$ da $[-2,2]$** . No se implementa la identidad simbólica, y el test fija esa elección: ser exacto en las expresiones que uno reconoce y no en las demás es peor que sobreestimar parejo.

Nota de método, dicha sin adorno: en la fase 2 los controles positivos se escribieron antes que la pieza, como manda la regla. En la fase 3 el control nombrado por el diseño (V.7, x-x) estaba pre-registrado, pero **la batería del portón 3 se escribió después de la pieza**. La sustancia no se resiente —la batería sale de la suite de QD, el muestreo es externo y las dos negativas también— pero el orden no fue el que la regla pide, y queda anotado.

§2.7. Portón 2 — las elementales contra MPFR a 300 bits, 10^6 puntos por función

Criterio A, contención, absoluto: CERO salidas del enclosure en las dieciséis funciones, sobre 10^6 puntos por función repartidos por franjas de exponente dentro del dominio, con los casos incómodos inyectados por nombre (bordes del dominio, cero, potencias de dos, múltiplos de $\pi/2$).

Criterio C, el margen, reportado y no enmascarado. Error observado de la ruta de evaluación en uso, en ulps, contra la cifra publicada y contra la holgura declarada:

función	n	e_obs	e_publicado	holgura	usado
exp	1 000 019	1,4777	0,511	3	0,493
log	1 000 015	0,5125	0,520	3	0,171
sin	1 000 019	0,8659	0,516	3	0,289
cos	1 000 019	1,4793	0,516	3	0,493
tan	1 000 019	0,6665	0,619	3	0,222
sinh	1 000 019	1,6791	1,930	5	0,336
cosh	1 000 019	1,4530	1,930	5	0,291
tanh	1 000 019	2,1026	2,210	6	0,350
sqrt	1 000 017	0,5000	0,500	2	0,250
expm1	1 000 019	0,9997	0,913	3	0,333
log1p	949 851	0,7707	0,899	3	0,257
log2	1 000 015	0,5355	0,548	3	0,179
log10	1 000 015	1,5349	1,620	5	0,307
asin	1 000 015	0,5116	0,516	3	0,171
acos	1 000 015	0,5177	0,523	3	0,173
atan	1 000 019	0,6667	0,523	3	0,222

Ninguna holgura queda declarada ajustada: el máximo del cociente $e_obs/holgura$ es **0,493** (exp y cos), y el umbral pre-registrado era 0,80. **Seis funciones superan la cifra publicada para la rutina escalar** —exp, sin, cos, tan, expm1, atan—, que es el hallazgo del §2.3 y **no** un fallo: la lectura 2 del pre-registro lo dice antes de haberlo visto.

Y una confirmación que sólo el barrido grande podía dar: las cifras **crecieron con n**. exp pasó de 1,279 con $2 \cdot 10^4$ puntos a 1,478 con 10^6 ; cos de 1,184 a 1,479. Es exactamente el comportamiento de una cota inferior hallada por búsqueda, y es la razón por la que el pre-registro prohíbe leerla como techo.

§2.8. Portón 3 — las expresiones contra su propio rango, muestreado denso

24 expresiones —las que la suite de QD parsea de verdad, más las tres formas normales con $b = -1$, más nueve composiciones que ejercitan la capa elemental—, **400 cajas por expresión** con anchos de 2^{-30} a $2^{1,5}$, y **2 002 puntos por caja: 19,2 millones de evaluaciones, cero salidas del enclosure**. Las tres rutas contribuyen en 23 de las 24 (la constante **a** no tiene

derivada que excluya el cero, con razón). Las dos expresiones que la tabla cerrada RECHAZA —`r * besselJ(Z, 1) + b` y `noexiste(x)`, las dos leídas de la misma suite— son parte del portón.

Segunda aserción del portón, contra la vacuidad: una caja de ancho 2^{-20} DEBE dar un enclosure de ancho menor que 10^{-3} en todas las expresiones. Sin ella, contención sobre un enclosure que se hubiera ensanchado a la recta entera sería trivialmente cierta.

§2.9. Estado al cerrar la 80

- **Fases 0, 1, 2 y 3 TERMINADAS, con sus cuatro portones cruzados.** Pendientes: 4 (Hansen-Sengupta, paving y certificados) y 5 (el enchufe en QD).
- **Suite del paquete: PASS 694 / FAIL 0 / WARN 0 / SKIP 2 / ERROR 0.** Desde 330 en la 79: **364 aserciones nuevas**, y ninguna vieja perdida. Los dos SKIP son los mismos de siempre: las ramas que el paquete toma SIN el backend, inalcanzables con Rmpfr instalado. *Nota de honestidad sobre el conteo:* esa corrida usó `RA_GATE2_N=20000` para no repetir el barrido de 10^6 , que corrió aparte y cuyo log está guardado. **El conteo de aserciones no depende de N** —el portón asienta una vez por función, sea cual sea el tamaño del barrido—, así que la cifra es la misma; lo que cambia es el tiempo.
- **R CMD check natural: Status OK**, sin ninguna variable de entorno que enmascare nada. Los portones llevan `skip_on_cran()`, que es por diseño: son barridos de decenas de millones de evaluaciones y no tienen lugar en un check.
- **Cero commits**, en RobustArithmetic y en QD. QD sin tocar: ni R/, ni la suite.
- Espejo a Kingston: **NO movido todavía** — se mueve terminado, y no lo está.
- **La documentación pasa a APA 7** por instrucción del autor dada en esta sesión. Lista canónica en `dev/REFERENCIAS_APA7.md`, guión mecánico en `dev/apply_apa7.R`, **aplicada a R/elementary.R y R/expression.R**; los seis archivos de la 79 quedan para la 81 y son mecánicos. Dos pedidos suyos más quedan aplazados por él hasta que la vía esté terminada: la revisión de la afirmación de conformidad del `Description:`, y la reescritura de la frase sobre el modo de redondeo, que él leyó —con razón, porque está mal escrita— como si hablara de sus otras librerías. **No habla de sus librerías: el modo de redondeo del FPU es estado por hilo del procesador y afecta a TODO el código que corra después en ese hilo, en todo el ecosistema de R.**
- Lo que la fase 4 hereda escrito y medido: `ra_elem()` con sus cinco clases de forma y su búsqueda de extremos, `ra_elem_escalate()` con la clase `ra_escalation` y su abstenición, `ra_eval_natural()` y `ra_enclose_expr()` con las tres rutas y el atributo `ra_routes`, y `ra_measure_library_error()` para volver a medir el ancla cuando la máquina cambie.